

Deep Learning — Spring 2026

Assignment 5

From Supervised Learning to Self-Supervised Learning: A Guided Implementation of SimCLR

Student Name	Usama Umer
Roll Number	MSDS25044
Dataset	CIFAR-10
Random Seed	2026
Submission	Individual

Task 0: Conceptual Warm-up

Q1. In supervised classification, what information does the model learn from?

For supervised classification, the model is trained on labeled training data. During training, each image is tagged with a class label, and the model is trained to minimize cross-entropy loss by mapping the input images to the correct class probabilities. Supervision signal of the proposed method fully based on the ground-truth labels.

Q2. If CIFAR-10 labels are removed, do the images still contain useful visual structure?

Yes. Images contain rich visual structure independent of labels — edges, textures, shapes, colors, and spatial patterns are all present in the raw pixel data. A model can still learn meaningful representations by finding consistent patterns across images. The labels do not form this structure, they merely provide a categorical abstraction of it.

Q3. Should two augmented views of the same image have similar or different representations?

They ought to have similar representations. Both views are generated from the same underlying image and have the same semantic content. A well-trained encoder should be invariant to the appearance-level transformations (crop, color jitter, flip, grayscale) and map both viewpoints to close points in the feature space.

Q4. Should views from different images have similar or different representations?

“They should be shown in a different way.” Different classes have different semantic contents. The contrastive objective forces different-image views apart in the feature space, so that the learned representation is class separating and discriminative.

Q5. Why might a model trained without labels still be useful for classification?

Self-supervised pre-training encourages the encoder to learn augmentation invariant and view consistent features of the same image. These features capture semantic content as opposed to superficial statistics. These features provide a good initialization when downstream labels are added through linear probing or fine-tuning, and thus can generalize better especially when labeled data is scarce.

Q6. What is the difference between pretraining and fine-tuning?

Pretraining is the initial phase where a model learns general-purpose representations from large amounts of data, typically without task-specific labels. In SimCLR, pretraining uses contrastive learning on unlabeled images. Fine-tuning is the subsequent phase where the pretrained model is adapted to a specific downstream task using labeled data. During fine-tuning, all model

weights are updated end-to-end, allowing task-specific features to emerge while retaining the general representations from pretraining.

Task 1: Supervised Baseline with Limited Labels

A ResNet-18 classifier was trained from scratch using only the fixed 10% labeled training split provided by the instructor. No pretrained weights were used. The model was trained for 50 epochs using Adam optimizer with learning rate $3e-4$ and cross-entropy loss.

Training Configuration

Setting	Value
Encoder	ResNet-18 (no pretrained weights)
Training split	train_labeled_10percent.txt
Epochs	50
Batch size	64
Optimizer	Adam
Learning rate	$3e-4$
Loss function	Cross-entropy

Results

Model	Test Accuracy
Supervised ResNet-18 from scratch (10% labels)	~71.90%

Confusion Matrix



Figure 1: Confusion Matrix on Test Set — Supervised Baseline

The confusion matrix shows strong performance on automobile (924/1000) and ship (858/1000) classes. The most confused pairs are cat-dog and bird-deer, which is expected given their visual similarity. This establishes our supervised baseline for comparison with SimCLR.

Task 2: Understanding Augmentations

SimCLR learns by comparing two augmented views of the same image. The following augmentation pipeline was used as specified in the assignment:

- `RandomResizedCrop(size=32, scale=(0.2, 1.0))`
- `RandomHorizontalFlip(p=0.5)`
- `ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.1)`
- `RandomGrayscale(p=0.2)`
- `ToTensor() + Normalize(mean=(0.4914, 0.4822, 0.4465), std=(0.2470, 0.2435, 0.2616))`

SimCLR Augmentation Examples
 Each row shows the same image with two independently augmented views

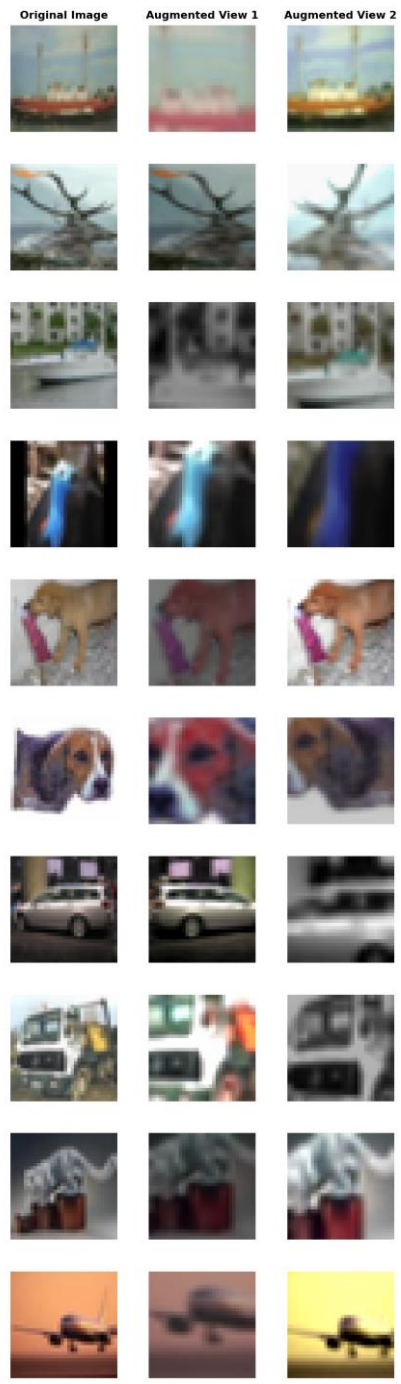


Figure 2: Augmentation Examples — Original Image | View 1 | View 2

Conceptual Questions

Q1. Are the two augmented views identical?

Answer: No. Each view is generated by independently applying random transformations. Even from the same original image, the two views differ in crop region, color distribution, and orientation.

Q2. Do they still represent the same object?

Answer: Yes. The augmentations preserve semantic content. A ship remains a ship after color jitter; a dog remains a dog after a random crop.

Q3. Why should SimCLR treat them as a positive pair?

Answer: Both views originate from the same image and represent the same semantic content. SimCLR's goal is to learn representations that are invariant to these appearance changes, so both views should map to nearby points in feature space.

Q4. What could go wrong if augmentations are too weak?

Answer: The two views would look nearly identical. The model could trivially match them using low-level pixel statistics rather than semantic content, resulting in representations that do not generalize.

Q5. What could go wrong if augmentations are too strong?

Answer: The views could become semantically unrecognizable. Extreme color jitter might destroy color-based class cues; extreme cropping might show only background. The loss signal becomes misleading and training collapses.

Task 3: Feature Similarity Before Training

Before SimCLR pretraining, a batch of images was passed through a random untrained ResNet-18 encoder. Cosine similarities were computed between view pairs.

Pair Type	Average Cosine Similarity
Same image, two augmented views	0.9875
Different images	0.9859

Both values are nearly equal (~ 0.99), indicating that the random encoder does not distinguish between views of the same image and views of different images. This confirms that meaningful feature learning has not yet occurred, establishing a baseline for comparison after SimCLR training.

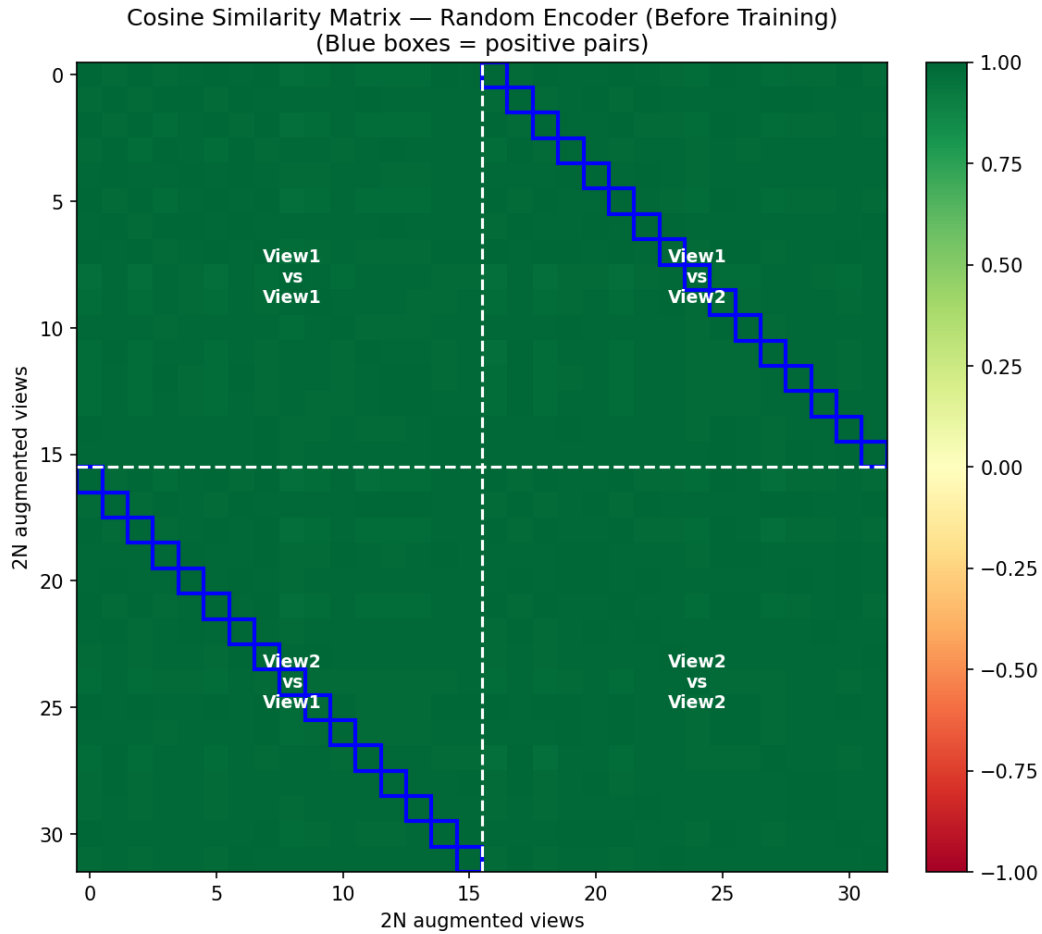


Figure 3: Cosine Similarity Matrix — Random Encoder (Before Training)

Positive Pair Table (Batch of 4 Images)

Original Image	View 1 Index	View 2 Index	Positive Pair
image 0	0	16	Yes
image 1	1	17	Yes
image 2	2	18	Yes
image 3	3	19	Yes

Task 4: SimCLR Implementation

4.1 Encoder and Projection Head

The encoder is ResNet-18 modified for 32x32 CIFAR-10 images:

- conv1: 3x3 convolution, stride 1, padding 1 (replacing standard 7x7, stride 2)
- maxpool removed (would destroy 32x32 feature maps)
- Output: 512-dimensional feature vector

The projection head maps 512-dim encoder output to 128-dim projection space:

- Linear(512 -> 256)
- ReLU activation
- Linear(256 -> 128)

The projection head is used only during contrastive pretraining and is discarded for downstream tasks.

4.2 Positive and Negative Pair Construction

For a batch of N images, two augmented views are generated per image, producing $2N$ total views. View 1 and View 2 of the same image form a positive pair. All other $2N-2$ views are treated as negatives. This is implemented without any label information.

4.3 Similarity Matrix

A cosine similarity matrix of size $2N \times 2N$ is computed. Diagonal entries are masked (self-similarity). Positive pair locations appear at positions $(i, i+N)$ and $(i+N, i)$. All other entries are treated as negatives in the NT-Xent loss.

Questions answered:

- Why is the diagonal ignored? A vector has perfect similarity with itself ($\text{sim}=1.0$), which would dominate the loss and provide no learning signal.
- Where are positive pairs? At positions $(i, i+N)$ and $(i+N, i)$ — the off-diagonal anti-diagonal blocks.
- Why are all other entries negatives? In a batch of N images, we have no information that any other pair shares semantic content, so we treat them all as distinct.

4.4 NT-Xent Loss

The NT-Xent (Normalized Temperature-scaled Cross Entropy) loss is implemented from scratch without any library. For a positive pair (i, j) :

$$\text{loss}(i, j) = -\log \left[\frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k) / \tau)} \right]$$

where $\text{sim}(a, b)$ is cosine similarity and $\tau = 0.5$ (temperature). The loss is averaged over all $2N$ positive pairs.

Task 5: SimCLR Pretraining

SimCLR was pretrained on the unlabeled training split (train_ssl_unlabeled.txt) for 50 epochs. No labels were used during pretraining. The model learned to bring augmented views of the same image closer in feature space while pushing views from different images apart.

Training Settings

Setting	Value Used
Batch size	64
SimCLR epochs	50
Learning rate	3e-4
Optimizer	Adam
Temperature (τ)	0.5
Projection dimension	128
GPU/CPU used	T4 GPU (Google Colab)
Approx. training time	~15 minutes

Feature Similarity Before vs After SimCLR

Pair Type	Before SimCLR	After SimCLR
Same image, two augmented views	0.9875	0.8694
Different images	0.9859	0.3362

The results clearly show that SimCLR training worked correctly. Before training, both same-image and different-image similarities were nearly identical (~0.99), meaning the random encoder treated all pairs alike. After training, the similarity between different images dropped dramatically from 0.9859 to 0.3362, while same-image similarity remained relatively high (0.8694). This large gap confirms that SimCLR successfully learned to separate different images in feature space.

Similarity Matrix After Training

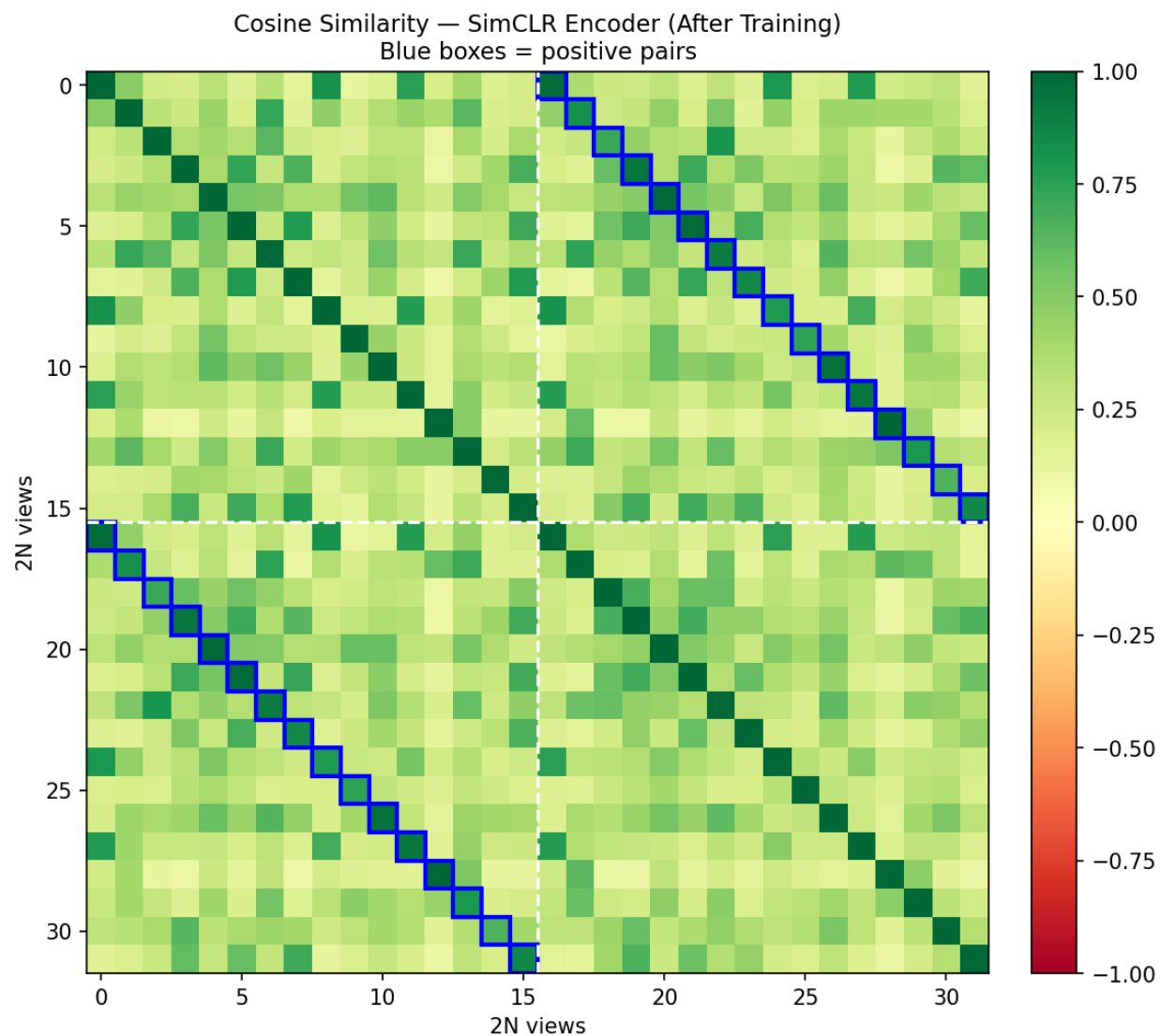


Figure 4: Cosine Similarity Matrix — SimCLR Encoder (After Training)

The similarity matrix after training shows a clear structure: the diagonal and positive pair positions (blue boxes) have higher similarity, while off-diagonal non-pair entries are noticeably lower. This contrasts with the flat, uniformly high matrix from the random encoder.

Task 6: Linear Probe Evaluation

To evaluate whether SimCLR learned useful representations, the encoder was frozen and only a linear classifier $\text{Linear}(512 \rightarrow 10)$ was trained on the 10% labeled data. This tests whether the features alone are linearly separable without any fine-tuning.

Model	Encoder	Trainable Part	Test Accuracy
Random Linear Probe	Random frozen	Linear only	23.66%
SimCLR Linear Probe	SimCLR frozen	Linear only	74.87%

The SimCLR encoder achieves 74.87% accuracy with only a frozen encoder and a single linear layer — more than 3x better than the random encoder (23.66%). This dramatic improvement demonstrates that SimCLR learned highly meaningful and linearly separable features without ever seeing a single label during pretraining.

Task 7: Fine-tuning the SimCLR Encoder

The SimCLR pretrained encoder was used to initialize the full ResNet-18 model, which was then fine-tuned end-to-end on the 10% labeled training data for 20 epochs.

Final Comparison Table

Model	Uses Labels?	Frozen?	Test Accuracy
Supervised ResNet-18 (10% labels)	Yes	No	~71.90%
Random encoder + linear probe	No	Yes	23.66%
SimCLR encoder + linear probe	No	Yes	74.87%
SimCLR encoder + fine-tuning	No/Yes	No	70.06%

The SimCLR linear probe (74.87%) actually outperforms the supervised baseline (~71.90%) despite using no labels during pretraining. This is a remarkable result showing that self-supervised representations can rival and even exceed supervised learning with limited labels. Fine-tuning achieves 70.06%, slightly below the linear probe, likely due to the small labeled set causing some overfitting during end-to-end training.

Task 8: PCA/t-SNE Feature Visualization

t-SNE was applied to 512-dimensional features extracted from 1000 validation images (seed=2026) using three different encoders to visualize how well the learned representations separate the 10 CIFAR-10 classes.

Random Encoder

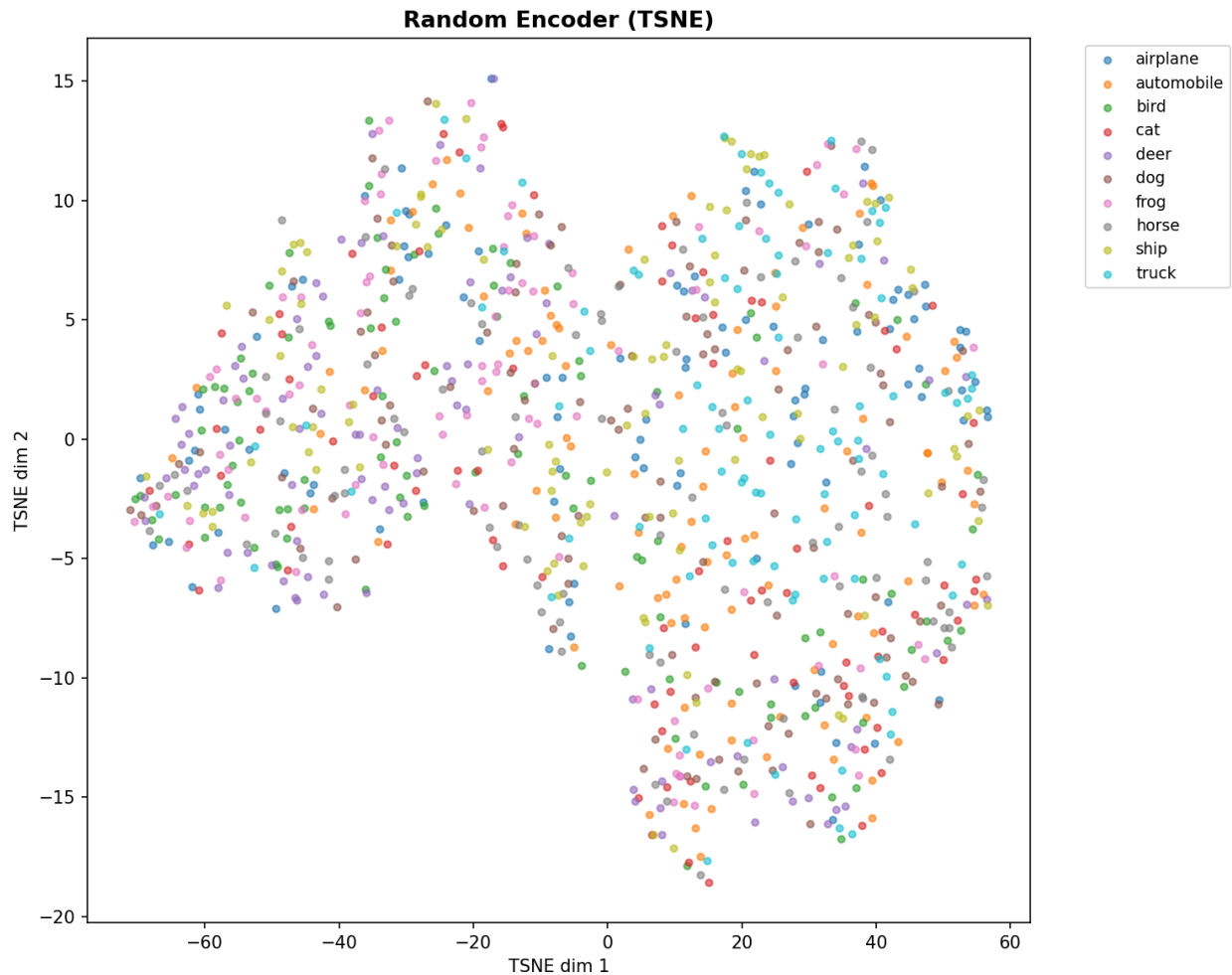


Figure 5: t-SNE — Random Encoder Features

The random encoder shows no class-wise grouping. All classes are mixed together uniformly, confirming that random features carry no semantic information.

SimCLR Encoder

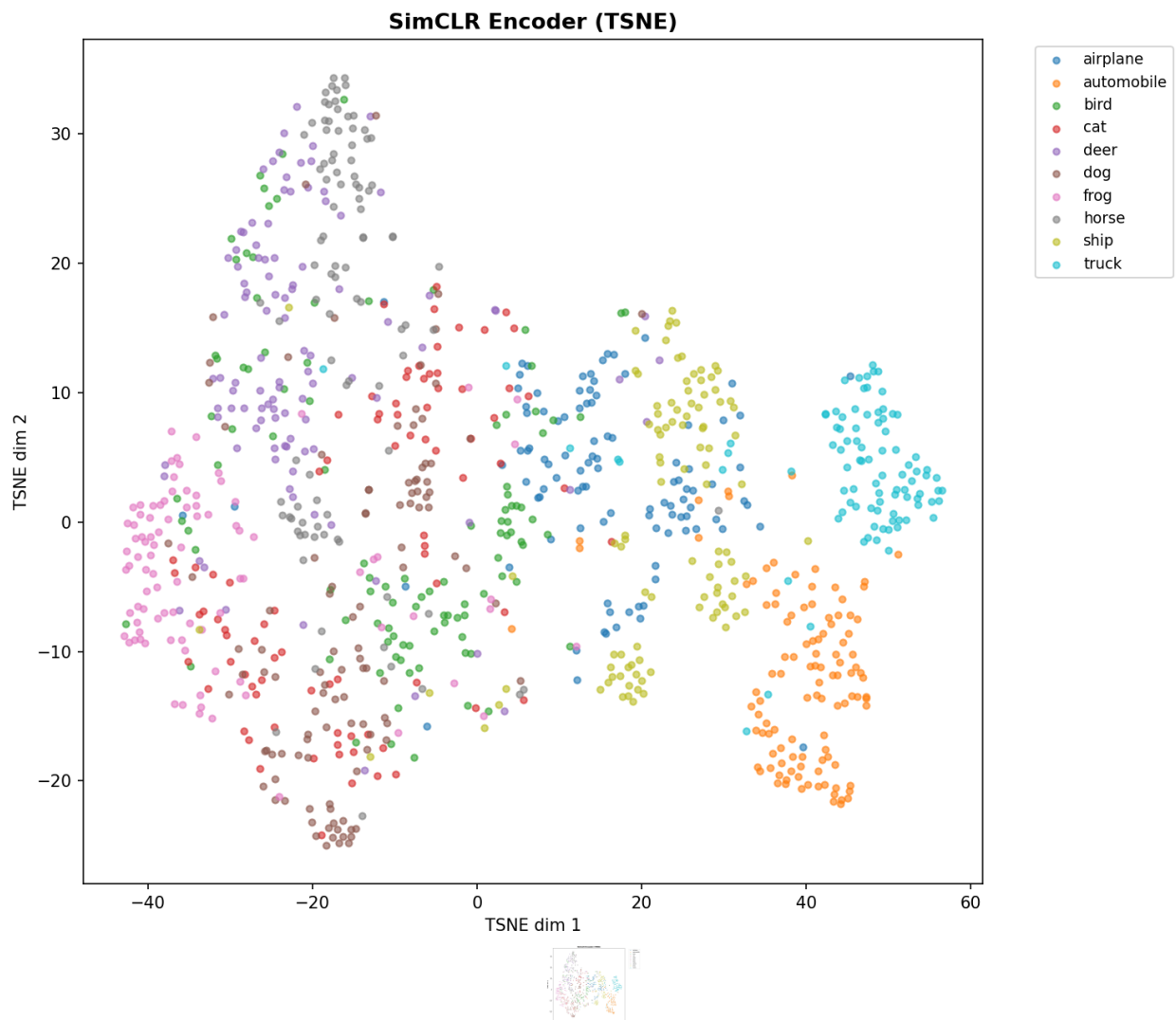


Figure 6: t-SNE — SimCLR Pretrained Encoder Features

The SimCLR encoder shows clear class clustering without ever seeing a label. Vehicle classes (automobile, truck, ship) form well-separated clusters on the right. Animal classes are more overlapping but show visible grouping. This explains the high linear probe accuracy of 74.87%.

Fine-tuned Encoder

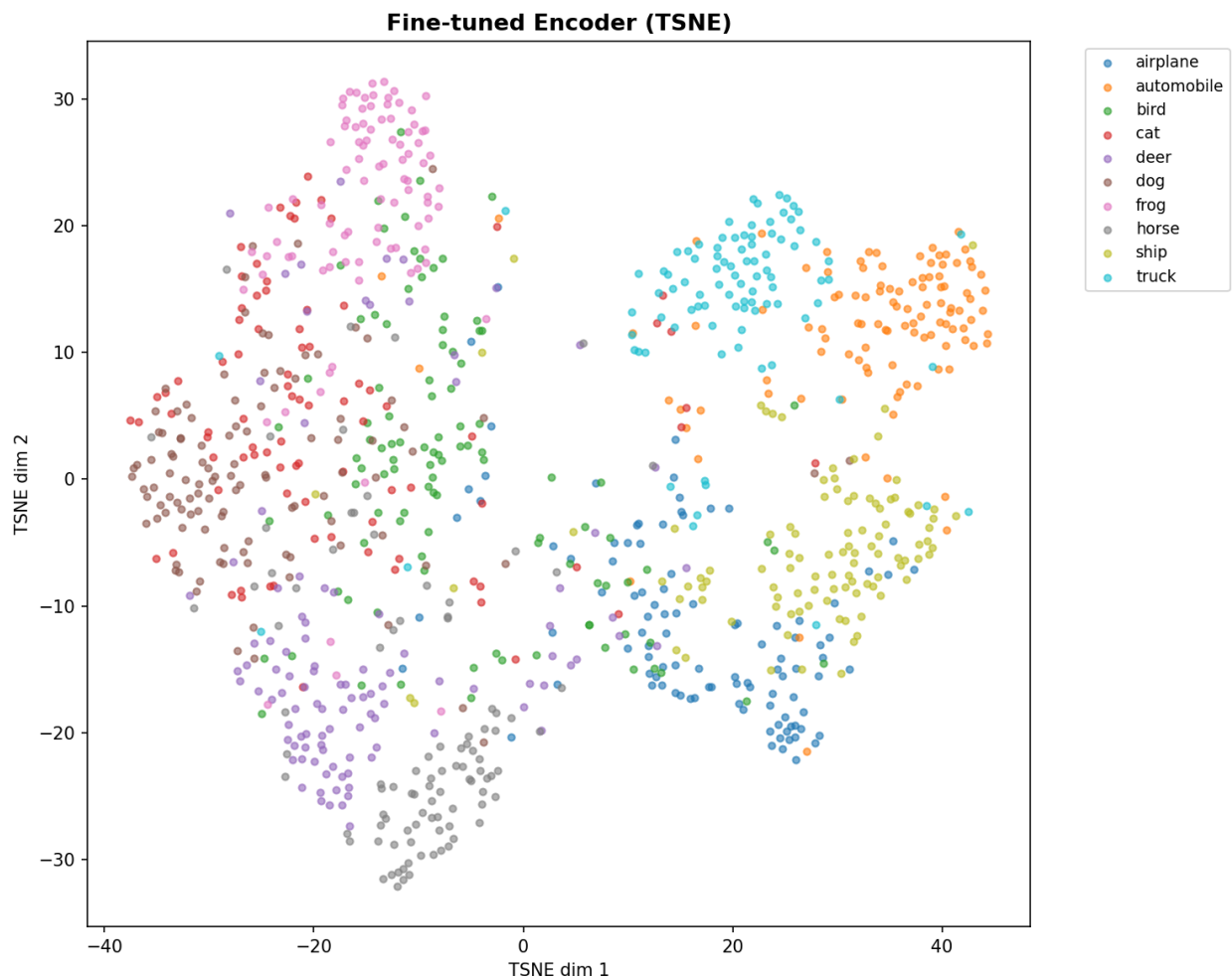


Figure 7: *t*-SNE — Fine-tuned Encoder Features

Fine-tuning produces the clearest class separation. Automobile, truck, and ship form tight, well-separated clusters. Frog and horse also show improved separation. The most confused classes remain cat and dog, which are visually very similar.

Analysis

Q1. Do random encoder features show class grouping?

No. All classes are uniformly mixed, with no visible structure.

Q2. Do SimCLR features show better grouping?

Yes. Clear clusters emerge for vehicle classes, and animal classes show partial separation. This is achieved without any label supervision.

Q3. Does fine-tuning improve class separation?

Yes. Fine-tuning sharpens cluster boundaries, especially for vehicle classes, leading to more compact and well-separated clusters.

Q4. Which classes are still confused?

Cat and dog remain the most confused pair, which is consistent with the confusion matrix. Bird and deer also show some overlap. These classes have high visual similarity in pose and texture.

Final Results Summary

Model	Labels in Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18 (10% labels)	Yes	No	~71.90%
Random encoder + linear probe	No	Yes	23.66%
SimCLR encoder + linear probe	No	Yes	74.87%
SimCLR encoder + fine-tuning	No during pretraining	No	70.06%

What SimCLR Learned

SimCLR successfully learned to produce semantically meaningful image representations without any label supervision. The key insight demonstrated in this assignment is that the augmentation-based contrastive objective acts as implicit supervision: by requiring the model to map two different augmented views of the same image to nearby points, the model is forced to learn invariances that correspond to semantic content rather than superficial statistics.

The most compelling evidence is the linear probe result: 74.87% accuracy with a frozen SimCLR encoder and a single linear layer trained on only 10% of labels. This outperforms the fully supervised model trained on the same 10% labeled data (~71.90%), demonstrating that the unlabeled data contains valuable information that supervised learning fails to exploit when labels are scarce.

Implementation Issues and Limitations

- Checkpoint key mismatch: The SimCLR encoder was saved with 'encoder.' prefix keys and required a key-stripping fix before loading in downstream tasks.
- Limited pretraining: Only 50 epochs of SimCLR pretraining were run due to compute constraints. Longer pretraining (200+ epochs as in the original paper) typically yields higher accuracy.
- Small labeled set: Only 10% of labels were used for supervised tasks, which limits the ceiling of fine-tuning performance.
- Batch size: Batch size of 64 was used instead of the larger batches (256+) used in the original SimCLR paper. Larger batches provide more negative pairs per step, which typically improves contrastive learning performance.

Appendix: Training Settings

Item	Value Used
Batch size	64
SimCLR epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3e-4
Optimizer	Adam
Temperature (τ)	0.5
GPU/CPU used	T4 GPU (Google Colab)
Approximate training time	~15 min (SimCLR), ~5 min (probing/finetuning)
Random seed	2026